Ensemble Learning and Random Forests

Voting, Bagging & Pasting, Random Forests, Boosting, Stacking

Anca Ralescu

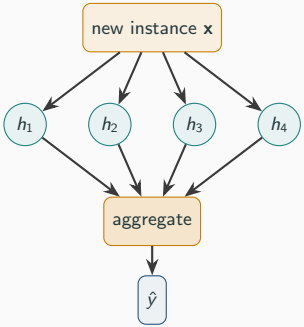
EECS Department, University of Cincinnati

Roadmap

1. Voting Classifiers
2. Bagging and Pasting
3. Random Forests
4. Boosting
5. Stacking
6. Summary

The Big Idea: the Wisdom of the Crowd

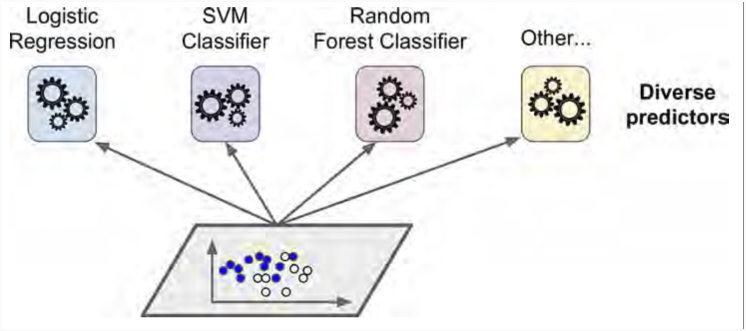
- An **ensemble** is a group of predictors whose outputs are **aggregated** into a single prediction.
- The aggregate is often better than the best individual member.
- Two ingredients make this work:
  - **enough** predictors
  - predictors that are **diverse** (make different errors)
- Families of methods covered:
  - Voting • Bagging / Pasting
  - Random Forests • Boosting
  - Stacking



Voting Classifiers

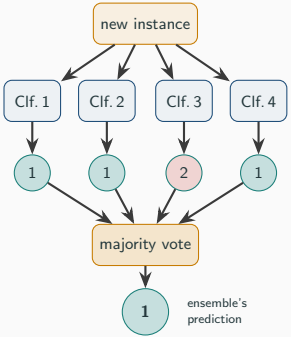
Training Diverse Classifiers

- Suppose we have trained several classifiers, each reaching about **80% accuracy**, e.g. Logistic Regression, SVM, Random Forest, k-NN, ...



Hard Voting

- **Hard voting**: each classifier casts one vote (its predicted class); the ensemble predicts the majority class.
- The hard-voting classifier often achieves higher accuracy than the best classifier in the ensemble.



Weak ⇒ strong

Even if each classifier is a **weak learner** (only slightly better than random guessing), the ensemble can be a **strong learner**, provided there are

- a sufficient number of weak learners, and
- they are sufficiently diverse.

Why Does It Work? A Slightly Biased Coin

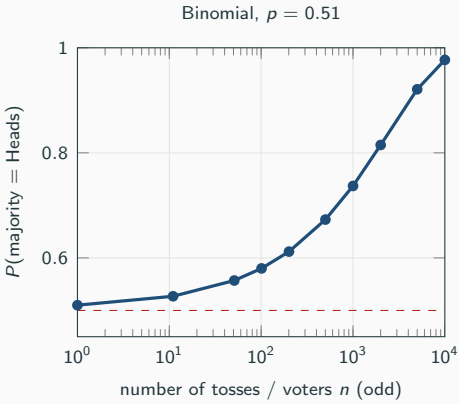
Let C be a biased coin; encode Head $= 1$, Tail $= 0$:

$$P(C = 1) = 0.51, \qquad P(C = 0) = 0.49$$

$$E(C) = 1 \cdot P(C = 1) + 0 \cdot P(C = 0) = 0.51$$

Toss C n times: about 510 heads and 490 tails per 1,000 tosses.

n tosses	$P(\#\text{Heads} > \#\text{Tails})$
1,000	≈ 0.75
10,000	≈ 0.97



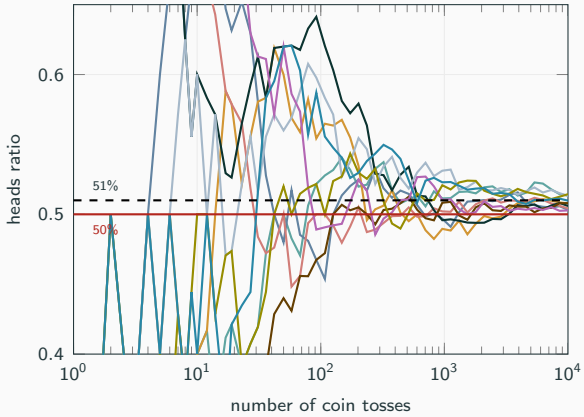
The Law of Large Numbers

Law of Large Numbers

As the number of tosses increases, the ratio

$$\frac{\#\,\text{heads}}{\#\,\text{tosses}} \longrightarrow P(C = \text{Head}) = 0.51$$

- 10 simulated series of biased-coin tosses.
- Eventually all series are so close to 0.51 that they stay above 0.50.



From Coins to Classifiers

- Replace the coin by a classifier: **Head** = correct classification, **Tail** = misclassification.
- Ensemble of **1,000 classifiers**, each correct with probability only 0.51 (barely better than random guessing).
- If they were *independent*, the majority-vote classifier could reach up to $\approx 75\%$ accuracy!

The catch: independence

- Coin tosses are **independent**; our classifiers are **not** — they are all trained on *the same data set*.
- They tend to make the *same types of errors* $\Rightarrow$ many majority votes go to the wrong class $\Rightarrow$ lower ensemble accuracy.

Hard vs. Soft Voting & Getting Diversity

Hard voting

Each classifier outputs a *class label*; predict the most frequent label.

Soft voting

Each classifier estimates *class probabilities*; average them and predict the class with the highest average probability.

$$\hat{y} = \underset{k}{\operatorname{argmax}} \frac{1}{N} \sum_{j=1}^{N} \hat{p}_j(k \mid \mathbf{x})$$

Very important note

Ensemble methods work best when the predictors are **as independent from one another as possible**.

How to get diverse classifiers

- Train them with **very different algorithms**
- Train them on **different training data**

This increases the chance that they make *very different* errors, improving the ensemble's accuracy.

Bagging and Pasting

Same Algorithm, Different Data

To obtain diverse classifiers, use the **same training algorithm** for every predictor, but train each on a **different random subset** of the training set.

Bagging (Breiman, 1996)

Bootstrap aggregating — sampling with replacement.

- an instance can be sampled several times *across* predictors
- an instance can be sampled several times *for the same* predictor

Pasting (Breiman, 1999)

Sampling without replacement.

- an instance can be sampled several times *across* predictors
- an instance cannot be sampled several times *for the same* predictor

Both training and prediction can be done in parallel.

Bagging vs. Pasting: Sampling Illustrated

Training set



Bagging (with replacement) — one predictor's sample:

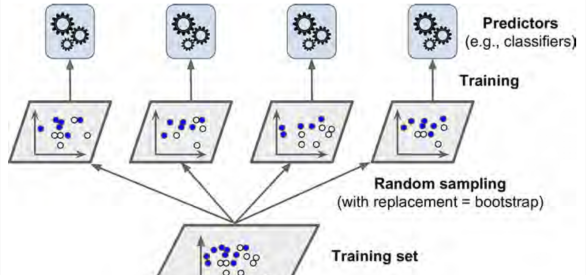


← "2" appears 3 times; 3, 4 never chosen

Pasting (without replacement) — one predictor's sample:



← each instance at most once (subset smaller than set)



Aggregation

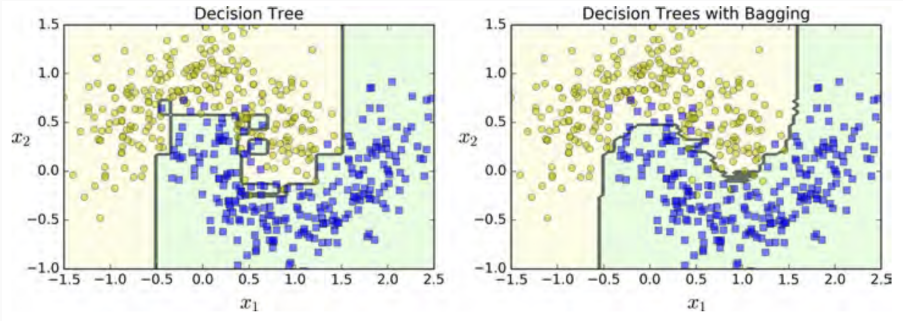
Once all predictors are trained, the ensemble predicts a new instance by **aggregating** the predictions of all predictors.

- **Classification**: the *statistical mode* — the most frequent prediction (just like hard voting).
- **Regression**: the *average* of the predictions.

Bias & variance

- Each individual predictor has a higher bias than if it were trained on the full original training set.
- Aggregation reduces both bias and variance.
- **Net result**: the ensemble has a *similar bias* but a **lower variance** than a single predictor trained on the original training set.

Single Decision Tree vs. Bagging Ensemble of 500 Trees



- The ensemble's decision boundary is much **smoother**: it generalizes better (comparable bias, less variance).

Bagging or Pasting?

- Bootstrapping introduces a bit **more diversity** in the subsets each predictor is trained on ⇒ bagging ends up with a slightly higher bias than pasting.
- But predictors end up **less correlated** ⇒ the ensemble's **variance is reduced**.
- Overall, bagging often results in better models ⇒ it is **generally preferred**.
- With enough time and CPU power: use **cross-validation** to evaluate both bagging and pasting and select the one that works best.

	Bagging	**Pasting**
Sampling	with replacement	without replacement
Diversity of subsets	higher	lower
Bias of ensemble	slightly higher	slightly lower
Correlation of predictors	lower	higher
Variance of ensemble	lower	higher

Random Patches, Random Subspaces, Extra-Trees

Sampling can be applied to **features** as well as to **instances**:

Random Patches

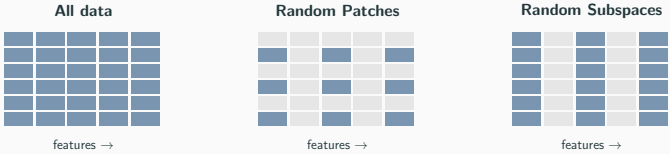
Random sampling of *both* training instances *and* features.

Random Subspaces

Random sampling of *features* only (all training instances kept).

Extremely Randomized Trees (Extra-Trees)

- Introduce further randomness: use random thresholds for each feature, rather than searching for the best thresholds.
- Much **faster to train**.



Random Forests

Random Forests

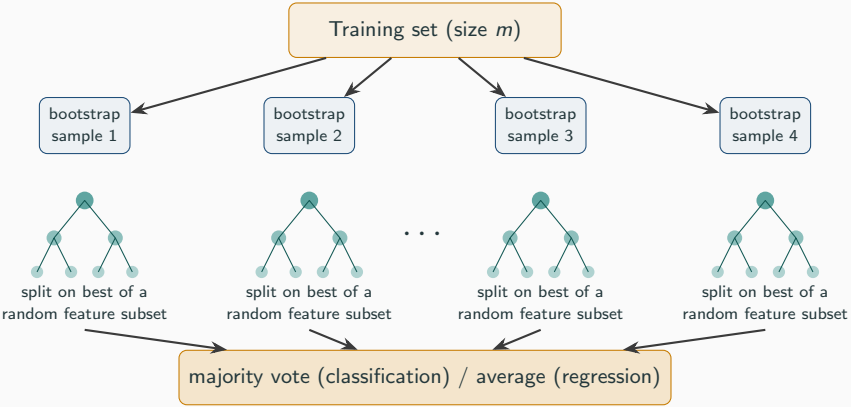
- An ensemble of **Decision Trees**, trained via the **bagging** method (or, sometimes, pasting).
- The number of samples per tree is usually the size of the whole training set.
- Introduces extra randomness when growing trees:
  - Instead of searching for the *very best* feature when splitting a node, it searches for the best feature among a **random subset of features**.
  - Resulting trees have **greater diversity**: trades a higher bias for a lower variance.
  - Yields an overall **better model**.

Random Forest or Extra-Trees?

It is hard to predict which (`RandomForestClassifier` or `ExtraTreesClassifier`) will perform better:

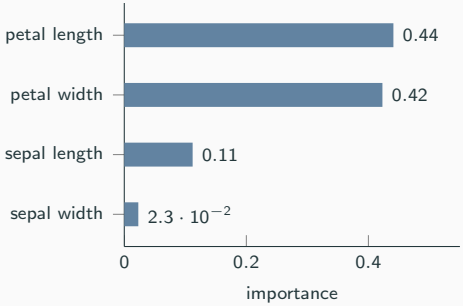
- try both and compare them using **cross-validation**;
- tune hyperparameters with **grid search** (evaluate all combinations of hyperparameters with cross-validation).

Random Forest at a Glance



Feature Importance

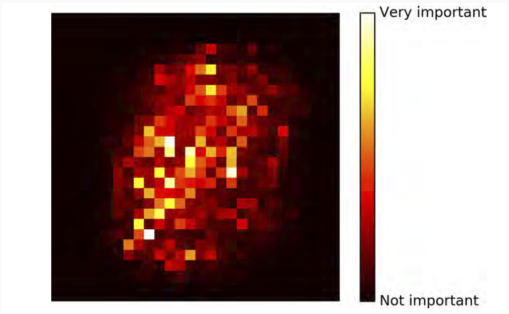
- In a decision tree, **important** features tend to appear *closer to the root*; unimportant ones appear closer to the leaves (or not at all).
- Use the **average depth** at which a feature appears across all trees of the forest to estimate its importance.
- Caveat: "importance" is a vague concept — information gain or other criteria come into play, as well as the *cost* associated with a feature.



Example — Iris dataset (Random Forest):

Feature Importance on MNIST

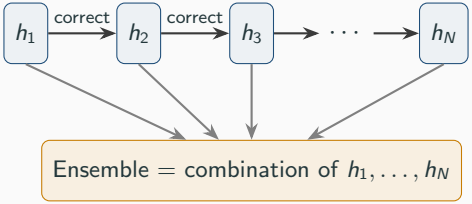
- MNIST: a set of handwritten digits (28×28 grayscale images, 60,000 training examples).
- Each pixel is a feature; a Random Forest gives each pixel an importance score.
- Pixels near the **centre** of the image matter most; border pixels are almost irrelevant.
- Random Forests are a handy tool for **feature selection**.



Boosting

Boosting (Hypothesis Boosting)

- Any ensemble method that can **combine several weak learners into a strong learner**.
- Idea: train predictors **sequentially**, each trying to *correct its predecessor*.
- Most popular:
  - **AdaBoost** (Adaptive Boosting)
  - **Gradient Boosting**

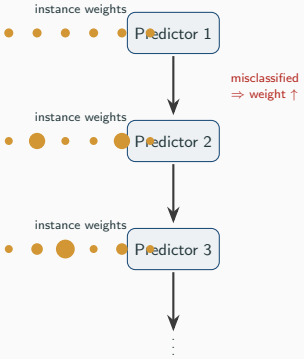


AdaBoost: Focus on the Hard Cases

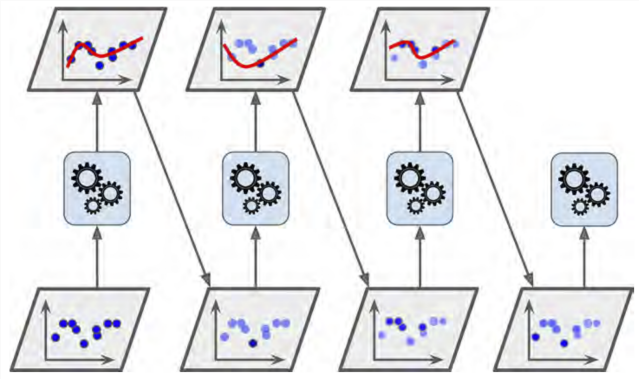
- Each new predictor corrects its predecessor by paying more attention to the training instances the predecessor **underfitted**.
- New predictors focus more and more on the hard cases.

Procedure

1. Train a first base classifier (e.g. a Decision Tree).
2. Make predictions on the training set.
3. Increase the relative weight of misclassified training instances.
4. Train the next classifier using the updated weights; predict on the training set; update weights; ...

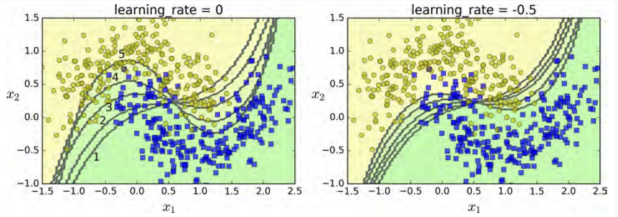


AdaBoost: Sequential Training with Weight Updates



Decision Boundaries of Consecutive Predictors (moons dataset)

- The 1st classifier gets many instances wrong $\Rightarrow$ their weights get **boosted**.
- The 2nd classifier does a better job on those instances, and so on.
- **Right plot**: same sequence of predictors, but the **learning rate is halved** (misclassified weights are boosted half as much at every iteration).



AdaBoost: Things to Note

Very important note

- Similar in spirit to **Gradient Descent** — but instead of tweaking a single predictor's parameters to minimize a cost function, AdaBoost **adds predictors** to the ensemble, gradually making it better.
- Once all predictors are trained, the ensemble predicts much like bagging/pasting, except that predictors have **different weights** depending on their overall accuracy on the weighted training set.
- **Drawback**: sequential learning cannot be parallelized (or only partially) — each predictor can only be trained *after* the previous one has been trained and evaluated. It does not scale as well as bagging or pasting.

The AdaBoost Algorithm (1/2): Error Rate

Training set of m instances; instance weights initialized to

$$w^{(i)} = \frac{1}{m}, \qquad i = 1, \ldots, m.$$

Weighted error rate of the j-th predictor:

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

numerator: total weight of the instances predictor j gets wrong

denominator: total weight

Predictor weight (η = learning rate, default 1):

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

- accurate predictor (r_j small) $\Rightarrow$ large α_j; random guessing ($r_j \approx 0.5$) $\Rightarrow \alpha_j \approx 0$; worse than random $\Rightarrow \alpha_j < 0$.

The AdaBoost Algorithm (2/2): Weight Update

For $i = 1, \ldots, m$:

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \quad \text{(correct: no change)} \\ w^{(i)}\, e^{\alpha_j} & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \quad \text{(incorrect: boost weight)} \end{cases}$$

Then **renormalize** so the weights add up to 1:

$$w^{(i)} \leftarrow \frac{w^{(i)}}{\sum_{k=1}^{m} w^{(k)}}$$

Loop

Train a new predictor on the re-weighted data, compute r_j, α_j, update the weights ... until the desired number of predictors is reached, or a perfect predictor is found.

AdaBoost Predictions

For a new instance $\mathbf{x}$, compute the predictions of all N predictors and weigh each by α_j:

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x})=k}}^{N} \alpha_j$$

- The predicted class is the one that receives the **majority of weighted votes**.

Mini example

Three predictors with $\alpha = (1.2,\ 0.4,\ 0.7)$ vote $(A,\ B,\ B)$.
Weighted votes: $A : 1.2, \quad B : 0.4 + 0.7 = 1.1 \quad \Rightarrow\ \hat{y} = A,$
even though B wins the *unweighted* vote.

Gradient Boosting

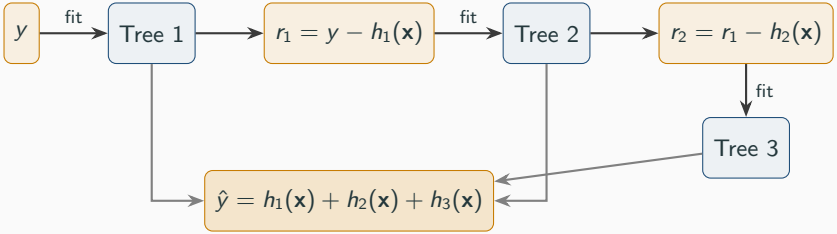
- Also works **sequentially**.
- Instead of re-weighting instances, each new predictor is fit to the residual errors made by the previous predictor.

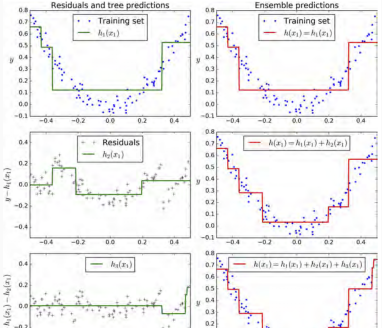
Gradient Tree Boosting / Gradient Boosted Regression Trees (GBRT)

1. Fit a decision tree to the training set (e.g. noisy quadratic data): $\text{pred}^{(1)}$.
2. Fit a 2nd tree to the residuals $y - \text{pred}^{(1)}$: $\text{pred}^{(2)}$.
3. Fit a 3rd tree to the residuals of the 2nd: $\text{pred}^{(3)}$.
4. Result: an ensemble of three trees,

$$\text{pred}(\mathbf{x}) = \sum_{i=1}^{3} \text{pred}^{(i)}(\mathbf{x})$$

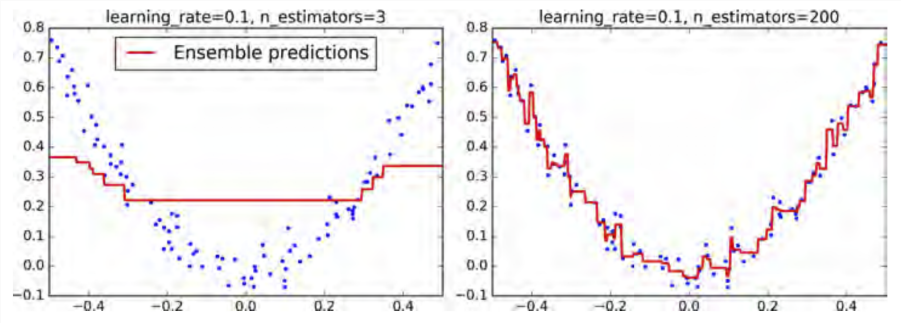
Gradient Boosting: Fitting the Residuals





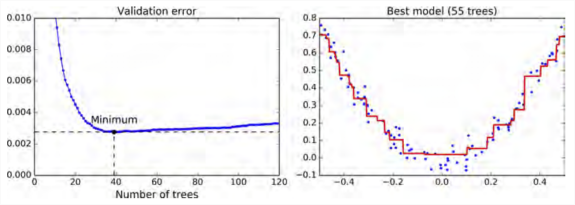
GBRT: How Many Trees?

- A key issue for GBRT is knowing **when to stop**.
- Too few trees ⇒ underfitting; too many trees ⇒ overfitting.



Early Stopping

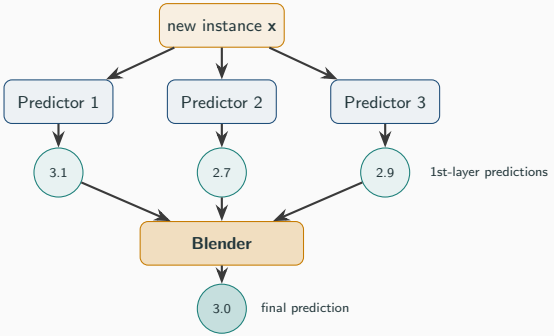
- Monitor the error on a **validation set** as trees are added.
- Find the number of trees at which the validation error is **minimal**.
- **Retrain** the ensemble using that (optimal) number of trees.



Stacking

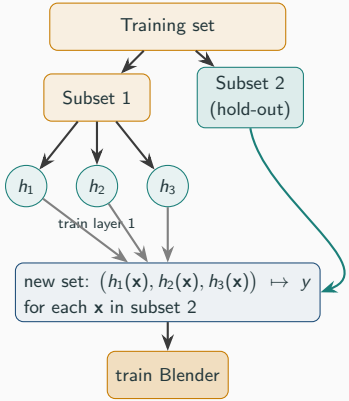
Stacking (Stacked Generalization)

- Idea: instead of using trivial (*ad-hoc*) functions such as hard voting to aggregate the predictions of an ensemble, **train a model to perform the aggregation**.
- This final model is called a **blender** (or **meta-learner**).

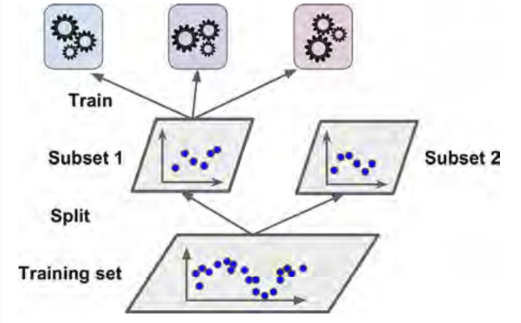


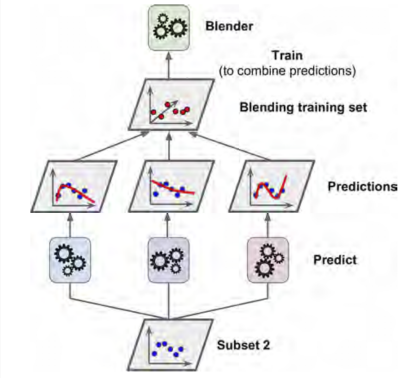
Training the Blender with a Hold-out Set

1. **Split** the training set into two subsets.
2. Use subset 1 to **train the first-layer** predictors.
3. Use them to make predictions on subset 2 (the **held-out** set):
   - predictions are "*clean*" — the predictors never saw these instances;
   - each held-out instance gets three predicted values.
4. Build a **new training set**: the three predictions are the input features (3-D), the original targets are kept.
5. **Train the blender** on this new set: it learns to predict the target from the first layer's predictions.



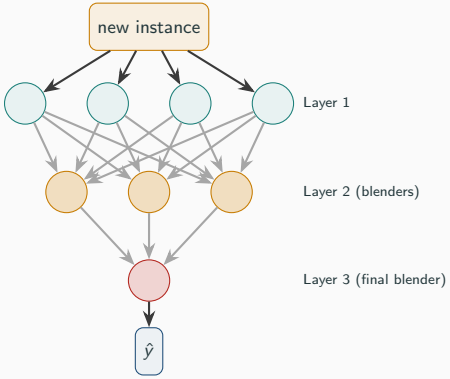
Training the First Layer & the Blender





Multi-layer Stacking

- Several different blenders can be trained (e.g. one Linear Regression, one Random Forest Regression, ...) $\Rightarrow$ a **layer of blenders**.
- Split the training set into **three** subsets:
  1. trains the **first layer**;
  2. builds the training set for the **second layer**, from first-layer predictions;
  3. builds the training set for the **third layer**, from second-layer predictions.
- A prediction for a new instance goes through each layer sequentially.



Summary

Summary

Method	How diversity / improvement arises	Aggregation	Parallel?
Voting	different algorithms	(weighted) majority / avg. prob.	yes
Bagging	bootstrap samples (w/ replacement)	mode / mean	yes
Pasting	samples w/o replacement	mode / mean	yes
Random Forest	bagging + random feature subset at each split	mode / mean	yes
Extra-Trees	+ random split thresholds	mode / mean	yes
AdaBoost	re-weight misclassified instances	α-weighted vote	no (sequential)
Gradient Boosting	fit residual errors	sum of predictors	no (sequential)
Stacking	learned combination	trained blender	per layer

Take-away

Ensembles turn many *diverse*, possibly weak predictors into a strong one — diversity is the key